

Turnkey 金库与闪兑多签操作手册

Turnkey 存款 → 清算 → 金库 资金流 与 闪兑/金库多签操作手册

适用系统：D3-FI 财库流水线（[supabase/functions/treasury](#) + [_shared/*](#)） 链：BNB Smart Chain（chainId [56](#)） 资产：USDT（[0x55d398326f99059fF775485246999027B3197955](#)，18 位小数） 托管：Turnkey（组织级 root quorum 多签 + Policy 授权）

本文分两部分：

1. **资金流架构**——钱包分几类、钱一步步怎么走、在哪里分流去「闪兑钱包」和「金库钱包」。
 2. **Turnkey Policy 与多签操作**——热钱包如何自动签名、金库/闪兑如何做多签、以及日常的批准/运维命令。
-

一、资金流架构总览



1.1 五类钱包及其角色

钱包类型	wallet_type	数量	是否需要后端自动签名	建议托管等级
存款地址	deposit	池化，默认目标 50	是（归集时从该地址转出）	热（HD 子账户）
存款 HD 母钱包	deposit_hd	1（D3-Deposits）	仅派生子地址，不签转账	热
清算钱包	settlement	默认 3	是（分流转出）	热
Gas 钱包	gas	1	是（给热钱包补 BNB 手续费）	热（只放少量 BNB）
闪兑钱包	flash_swap	1	是（付收益提现）	热·小额浮存
金库钱包	treasury	1	否（只收不发）	冷/多签

关键安全设计：后端 API 永远不需要、也不应该拥有从「金库」转出的签名权限。代码里金库只作为分流的收款终点（`settlement_to_treasury`），从不主动从金库发起转账。因此「金库出金」天然只能走人工多签，这正是我们要的效果。闪兑钱包是唯一需要后端自动付款的「热钱包」，所以它只留小

额浮存（默认清算额的 10%），把风险敞口降到最低。

1.2 完整资金流（对应代码）

1. 用户下单 → `createStakeIntent` (`deposit.ts`) 创建 `stake_intent`，从地址池 `claimDepositWalletFromPool` 领一个 `deposit` 子地址给用户。
2. 用户转账 USDT 到该地址 → 定时任务 `scanPendingDeposits` / `promoteDetectedDeposits` (`monitor.ts`) 在链上确认后把存款记录置为 `credited`。
3. 归集到清算钱包 → `enqueueDepositSweeps` 生成 `deposit_to_settlement` 任务，`pickSettlementWallet` 选负载最低的清算钱包，`processSweepJobs` 全额转出。转出前 `ensureGasBalance` 从 Gas 钱包给存款地址补 0.0005 BNB 手续费。
4. 分流 → `enqueueSettlementToTreasury`：当某清算钱包 USDT $\geq$ `SETTLEMENT_TO_TREASURY_MIN_USDT`（默认 100）时，按 `settlementFlashSwapSplitBps()`（默认 1000 bps = 10%）拆分：
 - `settlement_to_flash_swap`：10% → 闪兑钱包
 - `settlement_to_treasury`：其余 → 金库钱包
5. 闪兑钱包付收益 → 合伙人在前端提现，`requestPartnerYieldWithdraw` (`partnerYieldWithdraw.ts`) 建 `yield_flash_withdraw` 任务，从闪兑钱包把 USDT 打到合伙人钱包。
6. 所有转账都写 `sweep_jobs`、`ledger_entries`、`audit_log`，失败重试 3 次后转 `manual_review`。

整条流水线由一个函数编排：`runTreasuryPipeline` (`sweep.ts`)，通过 `POST /treasury/internal/run` 触发，pg_cron 每分钟跑一次。

二、Turnkey Policy 与多签操作

2.1 Turnkey 多签的运作原理（必须先懂这个）

Turnkey 的多签叫 Root Quorum（根法定人数） + Policy（策略）：

- Root Quorum：组织有一组「根用户」和一个 `threshold`（门槛）。任何没有被 Policy 明确放行的敏感操作（建钱包、签交易、改 quorum、建 Policy.....），都需要凑够 `threshold` 个根用户投票批准。
- Policy：可以对特定用户 / 特定钱包地址 / 特定交易条件单独放行（`EFFECT_ALLOW`）或拒绝（`EFFECT_DENY`），并可指定该操作所需的 `consensus`（共识表达式）。
- CONSENSUS_NEEDED：当后端 API 发起的活动需要凑票时，Turnkey 返回状态 `ACTIVITY_STATUS_CONSENSUS_NEEDED`。代码里 `isTurnkeyConsensusError()` 会捕获它，并把流程降级为「请在 Dashboard 手动建钱包 / 手动批准」。

本系统的目标配置是：

门槛 `threshold = 2`（人工双签兜底） + 一条 Policy 把「热钱包的 USDT/BNB 转账」单独放行给后端自动签名。于是：热钱包（存款/清算/Gas/闪兑）全自动跑；金库出金因为不在放行名单里，自动落回 2-of-N 人工多签。

2.2 角色与密钥规划

角色	类型	是否在 Root Quorum	用途
admin	人类 + Passkey	是	管理员，人工投票、建钱包、审批金库出金
it（联签人 / cosigner）	API Key	是	第二张共识票（Path C 自动凑票），必须是独立用户，不能和 admin 的 Passkey 挂在同一个用户下
d3-backend	API Key	建议不放进 quorum	后端自动签名用户，仅通过 Policy 放行热钱包

⚠ Path C 的坑（见 `turnkey-path-c.ts` 与 `turnkeyConsensus.ts` 的诊断逻辑）：如果后端 API Key 和 admin 的 Passkey 属于同一个 Turnkey 用户，那它无法充当「admin 批准后的第二票」。此时必须另配一把独立联签人的 API Key，写到：

```
TURNKEY_COSIGNER_API_PUBLIC_KEY=03b5fb7e...
TURNKEY_COSIGNER_API_PRIVATE_KEY=...
```

诊断脚本 `pathCReady` 判定为 true 的条件是：API 用户在 quorum 内、`threshold ≥ 2`、且该 API 用户没有 Passkey。

2.3 推荐的 Policy 设计

系统里的钱包地址都可以从环境变量拿到（清算、Gas、闪兑、金库都是固定地址；存款是 HD 池）。据此设计以下策略。

Turnkey 的 Policy DSL 字段（`eth.tx.*`、`wallet_account.address`、`approvers` 等）会随版本演进，落库前请对照 Turnkey 官方 Policy 参考核对字段名；下面给出的是意图 + 可直接改用的示例。

Policy 1 —— 放行后端自动签名「清算/闪兑」的 USDT 转出

允许 `d3-backend` 用户从清算钱包、闪兑钱包发起 BSC USDT 转账（覆盖 `settlement_to_treasury`、`settlement_to_flash_swap`、`yield_flash_withdraw`）。

```
{
  "policyName": "automation-hot-usdt-out",
  "effect": "EFFECT_ALLOW",
  // 单签即可（仅后端这一票），绕过 2-of-N 门槛
  "consensus": "approvers.any(user, user.id == '<D3_BACKEND_USER_ID>')",
  "condition": "eth.tx.chain_id == 56 && eth.tx.to == '0x55d398326f99059fF775485246999027B31979'",
  "notes": "后端自动签名：清算/闪兑钱包的 USDT 转出。金库地址不在此名单，出金落回人工多签。"
}
```

Policy 2 —— 放行后端归集「存款 → 清算」

存款地址是 HD 池、动态派生，不便逐个枚举。用收款方约束更稳：凡是 USDT 且收款方是清算钱包，就放行。

```
{
  "policyName": "automation-deposit-sweep",
  "effect": "EFFECT_ALLOW",
  "consensus": "approvers.any(user, user.id == '<D3_BACKEND_USER_ID>')",
  // 收款方 (ERC20 transfer 的 to 参数) 必须是我们的清算钱包之一
  "condition": "eth.tx.chain_id == 56 && eth.tx.to == '0x55d398326f99059fF775485246999027B31979'",
  "notes": "后端自动签名：任意存款地址把 USDT 归集到清算钱包。"
}
```

Policy 3 —— 放行 Gas 钱包补手续费（原生 BNB，带限额）

```
{
  "policyName": "automation-gas-topup",
  "effect": "EFFECT_ALLOW",
  "consensus": "approvers.any(user, user.id == '<D3_BACKEND_USER_ID>')",
  // 仅 Gas 钱包、原生转账、单笔 ≤ 0.01 BNB（代码单次补 0.0005）
  "condition": "eth.tx.chain_id == 56 && wallet_account.address == '<GAS_ADDR>' && eth.tx.value <= 0.01",
  "notes": "后端自动签名：Gas 钱包给热钱包补 BNB，单笔限额兜底。"
}
```

Policy 4（可选）—— 闪兑钱包出金限额（纵深防御）

即使 Policy 1 已放行闪兑，也可再加一条金额上限，把被盗风险锁在浮存额度内。

```
{
  "policyName": "flash-swap-cap",
  "effect": "EFFECT_ALLOW",
  "consensus": "approvers.any(user, user.id == '<D3_BACKEND_USER_ID>')",
  "condition": "wallet_account.address == '<FLASH_SWAP_ADDR>' && eth.tx.contract_call_args['am
  "notes": "闪兑单笔提现上限 500 USDT, 超额需人工。"
}
```

金库 (treasury) —— 不写任何放行 Policy

金库地址不出现在上述任何 `EFFECT_ALLOW` 里。于是：

- 只进不出 (自动部分)： `settlement_to_treasury` 是「转入」金库，由清算钱包签名，Policy 1 放行的是清算钱包，转入金库完全正常。
- 出金 = 人工多签：任何想「从金库转出」的活动都不被放行，自动落回 Root Quorum `threshold = 2`，必须由 `admin` + `it` (或另一名根用户) 在 Turnkey Dashboard 双签批准。这就是金库多签的落地方式。

如需更强隔离，见 2.6 「方案 A：金库用独立多签地址」。

2.4 两种落地方案对比

	方案 A (推荐·最简单)	方案 B (全 Turnkey Policy)
金库	独立多签地址 (另一个高门槛 Turnkey 钱包 / Gnosis Safe), 后端只登记地址、无签名权	与热钱包同组织, 靠「不写放行 Policy」+ <code>threshold≥2</code> 实现出金多签
配置	只需 <code>TURNKEY_TREASURY_ADDRESS</code> (可不填 <code>WALLET_ID</code>)	需完整配 Root Quorum + 上面 4 条 Policy
优点	私钥物理隔离, 最难出事	全部在一个组织, 运维统一
缺点	金库出金要在另一处操作	Policy 写错=风险, 需仔细核对 DSL

代码对两种都支持：`registerExternalWallet` 里 `provider: 'external'` (无 `walletId` 时) 即方案 A；填了 `TURNKEY_TREASURY_WALLET_ID` 且同组织即方案 B。

2.5 环境变量清单 (Edge Function Secrets)

```
# — Turnkey 组织与后端 API 用户 —
TURNKEY_ORGANIZATION_ID=...
TURNKEY_API_PUBLIC_KEY=...           # d3-backend 公钥
TURNKEY_API_PRIVATE_KEY=...          # d3-backend 私钥

# — Path C 联签人 (独立用户, 用于自动凑第二票) —
TURNKEY_COSIGNER_API_PUBLIC_KEY=...
TURNKEY_COSIGNER_API_PRIVATE_KEY=...

# — 金库 (只进不出; 方案 A 可只填地址) —
TURNKEY_TREASURY_ADDRESS=0x...
TURNKEY_TREASURY_WALLET_ID=...       # 方案 B 才需要

# — 闪兑钱包 —
TURNKEY_FLASH_SWAP_WALLET_ADDRESS=0x...
TURNKEY_FLASH_SWAP_WALLET_ID=...

# — 清算 / Gas (Policy 阻断自动建钱包时, 手动建好再登记) —
TURNKEY_SETTLEMENT_ADDRESSES=0x...,0x...,0x...
TURNKEY_SETTLEMENT_WALLET_IDS=uuid1,uuid2,uuid3
TURNKEY_GAS_WALLET_ADDRESS=0x...
TURNKEY_GAS_WALLET_ID=...

# — 存款 HD 母钱包 (可留空自动建) —
TURNKEY_DEPOSITS_WALLET_ID=...
TURNKEY_DEPOSITS_WALLET_ADDRESS=0x...

# — 流水线参数 —
SETTLEMENT_WALLET_COUNT=3
SETTLEMENT_TO_TREASURY_MIN_USDT=100  # 触发分流的最低余额 (测试用 1)
SETTLEMENT_TO_FLASH_SWAP_PCT=10      # 分流去闪兑的百分比
DEPOSIT_POOL_TARGET_SIZE=50
DEPOSIT_POOL_MIN_AVAILABLE=10
DEPOSIT_POOL_BATCH_SIZE=10
TREASURY_CRON_SECRET=...              # cron 与 admin 接口鉴权
```

设置方式: `npx supabase secrets set KEY=VALUE` (切勿加 `VITE_` 前缀, 切勿进 Netlify)。

2.6 操作教程 (从零到跑起来)

Step 0 — 在 Turnkey Dashboard 建组织与用户

1. 建组织; 把 `admin` (Passkey) 设为根用户。
2. 建独立用户 `it` 并生成 API Key → 填 `TURNKEY_COSIGNER_API_*`。
3. 建后端用户 `d3-backend` 并生成 API Key → 填 `TURNKEY_API_*`。
4. 把 Root Quorum 门槛设为 2 (成员含 `admin`、`it`)。

Step 1 — 建热钱包与金库

- 清算 ×3、Gas ×1、闪兑 ×1、金库 ×1，在 Dashboard 建好（或让后端自动建，若被 `CONSENSUS_NEEDED` 拦住就手动建）。
- 把各地址/ `walletId` 填进 2.5 的环境变量。

Step 2 — 建上面 4 条 Policy（`ACTIVITY_TYPE_CREATE_POLICY`）

- 把示例里的 `<D3_BACKEND_USER_ID>`、各 `<...ADDR>` 换成真实值。
- 建 Policy 本身也可能需要 quorum 批准（见 Step 4）。

Step 3 — 预生成存款地址池

```
npm run treasury:bootstrap-pool      # 目标 50
npm run treasury:bootstrap-pool -- 100 # 目标 100
```

底层调 `POST /treasury/admin/bootstrap-deposit-pool`，用 HD 母钱包 `createWalletAccounts` 批量派生。

Step 4 — 处理 `CONSENSUS_NEEDED`（Path C 自动凑票）

```
npm run treasury:path-c      # 诊断：谁在 quorum、有哪些待批活动、是否 pathCReady
npm run treasury:path-c -- approve # 用联签人/后端 API Key 投出第二票
```

或走 Edge 接口（需 `TREASURY_CRON_SECRET`）：

```
# 查看共识状态
curl -H "X-Treasury-Cron-Secret: $TREASURY_CRON_SECRET" \
  "$SUPABASE_URL/functions/v1/treasury/admin/turnkey/consensus-status"
# 批准全部可批准活动
curl -X POST -H "X-Treasury-Cron-Secret: $TREASURY_CRON_SECRET" \
  "$SUPABASE_URL/functions/v1/treasury/admin/turnkey/approve-consensus"
```

典型两票流程：`admin` 在 Dashboard 投第一票 → 后端/联签人 API `approve-consensus` 投第二票 → 活动执行。

Step 5 — 登记基础设施钱包并自检

```
curl -X POST -H "X-Treasury-Cron-Secret: $TREASURY_CRON_SECRET" \
  "$SUPABASE_URL/functions/v1/treasury/admin/bootstrap"
curl "$SUPABASE_URL/functions/v1/treasury/health" # 看 turnkey/treasury/infra 状态
```

Step 6 — 开定时流水线（每分钟）

```
npm run treasury:setup-cron
```


它会：写 `TREASURY_CRON_SECRET` 到 secrets 与 `.env` → 建 pg_cron 任务 `d3-treasury-pipeline` (`*/1 * * * *`，打 `POST /internal/run`) → 跑一次 bootstrap + 测试。

手动触发一次：

```
curl -X POST -H "X-Treasury-Cron-Secret: $TREASURY_CRON_SECRET" \
  "$SUPABASE_URL/functions/v1/treasury/internal/run"
```

2.7 金库出金（人工多签）标准流程

1. 在 Turnkey Dashboard 对金库钱包发起 `Sign Transaction` (USDT transfer 到目标地址)。
2. 因为没有放行 Policy，活动进入 `CONSENSUS_NEEDED`。
3. `admin` 投票批准；`it`（或另一名根用户）投第二票，凑满 `threshold = 2`。
4. 活动执行、上链。全过程有 Turnkey 审计记录；如走系统接口也会进 `audit_log`。

建议：金库出金**始终人工**，不要为图省事给金库加放行 Policy——那会让「金库多签」形同虚设。

2.8 日常运维与排障

- 健康检查：`GET /treasury/health` —— 看 `turnkey`、`treasuryAddress/WalletId`、`infrastructure`（各类钱包计数、排队 sweep 数、已入账存款数）、`depositPool`。
 - 地址池将空：`replenishDepositPoolIfLow` 在流水线里自动补；低于 `DEPOSIT_POOL_MIN_AVAILABLE` (10) 触发。
 - sweep 失败：重试 3 次后置 `manual_review`，在 `sweep_jobs` 表查 `error_message`。收益提现失败会把 `partner_yield_withdrawals` 置 `failed / manual_review`。
 - 闪兑钱包余额不足：调高 `SETTLEMENT_TO_FLASH_SWAP_PCT`，或手动补 USDT 进闪兑地址。
 - Gas 不足：给 Gas 钱包充 BNB；单笔补 0.0005 BNB，阈值 0.0005 BNB（`turnkey.ts` 常量）。
 - `CONSENSUS_NEEDED` 卡住：先跑 `npm run treasury:path-c` 诊断，按 `recommendations` 处理（多半是联签人未配、或 API Key 和 Passkey 同用户）。
-

附：关键代码位置

功能	文件
Turnkey 签名 / 建钱包 / 分流参数	supabase/functions/_shared/turnkey.ts
多签诊断与自动批准	supabase/functions/_shared/turnkeyConsensus.ts
钱包登记 (含 treasury/flash_swap)	supabase/functions/_shared/wallets.ts
归集 + 分流 + 流水线编排	supabase/functions/_shared/sweep.ts
存款地址池 / HD 派生	supabase/functions/_shared/depositPool.ts 、 depositsHd.ts
收益提现 (闪兑钱包付款)	supabase/functions/_shared/partnerYieldWithdraw.ts
HTTP 路由 / admin 接口	supabase/functions/treasury/index.ts
Path C 诊断/批准脚本	scripts/turnkey-path-c.ts
建池脚本 / 建 cron 脚本	scripts/treasury-bootstrap-deposit-pool.ts 、 treasury-setup-cron.ts